DECENTRATHON 6.0 · SOUP KERNEL CHALLENGE

Обучение быстрее и в 5 раз легче

Свои Triton-кернелы для soup train на бесплатной Kaggle T4

1,80×
быстрее: 78 → 44 с

5,17×
меньше памяти: 14 345 → 2 775 МиБ

10 / 10
проверок, eval loss = база

Задача

Фиксированный запуск soup train

- Qwen2.5-1.5B-Instruct, веса 4-бит NF4, LoRA r=16 на q_proj и v_proj
- batch 4, до 512 токенов, 256 казахских инструкций, 64 шага
- Kaggle T4: 15 ГБ, fp16, без bf16
- Меряют полное время команды, пик памяти по nvidia-smi и качество адаптера
- 10 проверок: та же задача, те же метрики, rewind.jsonl, тесты Soup

БАЗА, SOUP 0.77.0

82 с

весь запуск, из них ~56 с — 64 шага

14 345 МиБ

пик памяти из 15 360

Куда уходят время и память у базы

Логиты на 152 000 токенов

lm_head в fp32, логиты [4, T, 151 936] и их копии для loss, энтропии, точности и rewind — несколько гигабайт

4-битные веса

bitsandbytes распаковывает каждый вес дважды за шаг и хранит входы замороженных слоёв

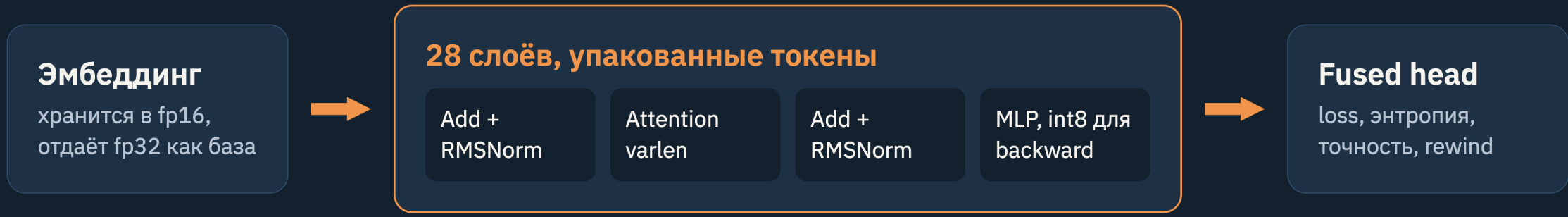
Паддинг

Строка в среднем ~220 токенов (медиана 140), но каждую добивают до самой длинной в батче

Накладные расходы

~44 % GPU — мелкие операции и приведения типов; ~20 с от старта до первого шага

Свой шаг обучения на реальных токенах



Без паддинга. При правом паддинге каждый реальный токен видит те же ключи на тех же позициях. Строки разных батчей не смешиваются.

Свой backward. Каждый блок хранит только то, что нужно его обратному проходу. Подключение — одна функция apply().

Fused head: полных логитов нет

1. Строки по 128 → GEMM fp16 на тензорных ядрах, логиты fp32 только для этого куска
2. **head_row_stats_kernel**: один проход по 151 936 логитам — онлайн max, Σexp, энтропия, argmax
3. Для размеченных строк второй проход пишет softmax − onehot на месте; градиент — ещё одна GEMM
4. Энтропия, точность и потери строк уходят в метрики TRL и rewind Soup по их же формулам

ЛОГИТЫ НА ПИКЕ

несколько ГБ

↓

78 МБ

один кусок 128 × 151 936 fp32;
эмбеддинг/lm_head хранится в fp16

Идея — Liger-Kernel и Cut Cross-Entropy (Apple), online softmax — Milakov & Gimelshein; код наш.

4-битные веса и MLP

`nf4_unpack_kernel`

Распаковка NF4 бит в бит

- Один запуск вместо двух у bitsandbytes
- Поток читает 8 значений, пишет 16 байт одной инструкцией
- Сразу в общий буфер q|k|v или gate|up — одна GEMM
- mul_rn: без FMA, ровно как bitsandbytes (тест: torch.equal)

`gated_silu_q8_kernel`

MLP: int8 для backward

- silu(g)·u из точных fp16 — forward и loss не меняются
- g|u сохраняются в int8, одна шкала на 128 значений
- 35 → 18 КБ на токен на слой, −680 МБ на пике
- Шум градиента ~0,8 % на элемент; кривая loss = база

Attention без паддинга

Один вызов на слой

Memory-efficient attention PyTorch в режиме переменной длины: каждая строка на своей длине, causal внутри строки

Храним упакованное

Для backward — повёрнутые q|k|v с 2 kv-головами вместо 12 повторённых и выход, только реальные токены

RoPE + LoRA одним кернелом

rope_lora_kernel добавляет выходы LoRA и поворачивает q, k в fp32 на месте; **rope_inverse_kernel** — обратно

Те же маски dropout

Те же вызовы генератора CUDA, что у peft; в backward повторяем его состояние, а не храним маски

Итог: кривая loss совпадает с базой до 0,01 %, eval loss — до 0,002 %

Старт и накладные расходы

7,9 → 0,5 с

Первый шаг

Пока Soup грузит модель, фоновый поток компилирует кернелы, собирает их C-модули запуска (gcc) и подгружает cuBLAS и attention

−3,5 с

Отпечаток datasets

.map() хешировал токенизатор медленным pickle на Python; теперь SHA-256 его полного JSON. На выходы Soup не влияет

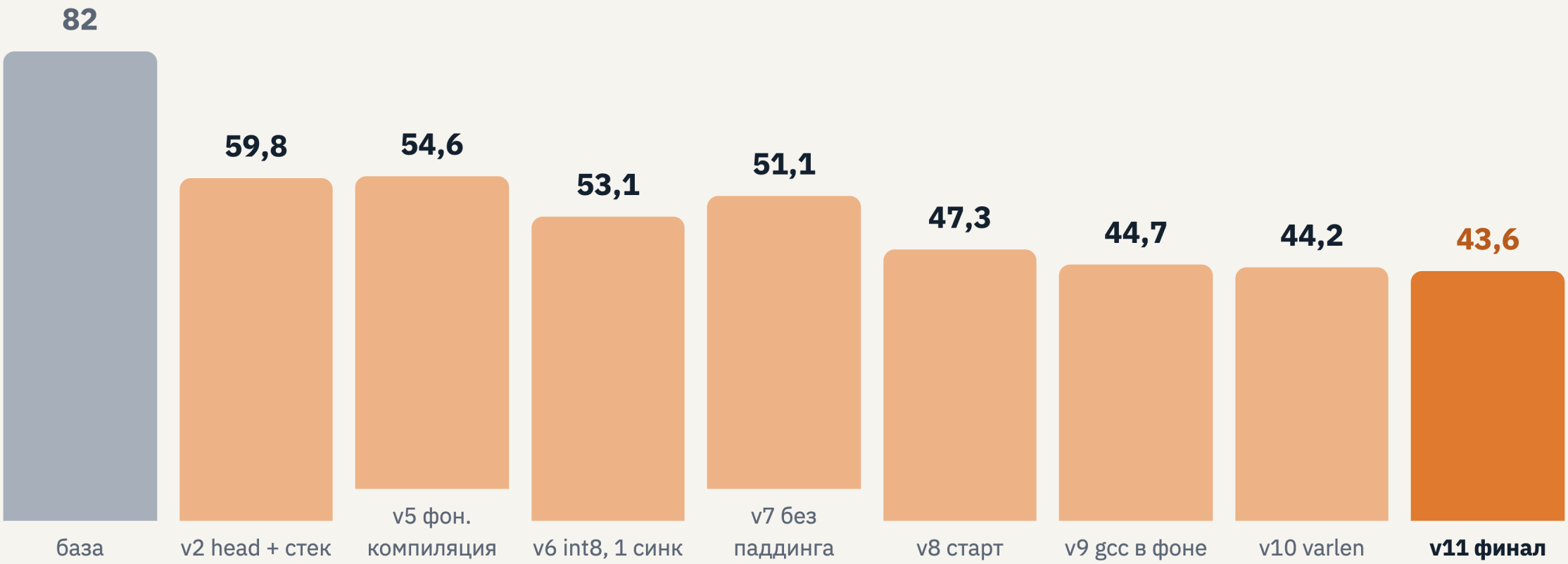
1 синк

на шаг вместо пяти

Индексы упаковки за одну передачу с GPU; fused backward нормы; константы слоёв и fp16-копии LoRA — один раз

Всё нашли профилем: torch.profiler по кернелам, cProfile всего процесса, таймлайн по этапам

Путь: время полного запуска, с



Прогоны на Kaggle T4 (база в тех же сессиях 75–82 с); финал — --repeats 2, база 78,4 с. Память по пути: 4 021 → 2 775 МиБ.

Результат по скриптам жюри

Метрика	База	Мы	Отношение
Время soup train, с	78,4	43,6	1,80×
Пик памяти, МиБ	14 345	2 775	5,17×
Eval loss	2,04962	2,04967	+0,002 %
Доля наших кернелов	—	19,3 %	порог 5 %

10 из 10 проверок: та же задача и выходы Soup, кривая loss в пределах 0,02 %, чистый замер (пик nvidia-smi совпадает с PyTorch), 79 тестов Soup.

run_task.py --repeats 2: база, решение, решение, база — как у жюри

10 своих Triton-кернелов в обучении

Кернел	Что заменяет
head_row_stats_kernel	log_softmax, CE, энтропия, argmax по полным логитам
nf4_unpack_kernel	распаковка NF4 bitsandbytes (два запуска)
rmsnorm_fwd_kernel	Qwen2RMSNorm + сложение с residual
add_rmsnorm_bwd_kernel	их backward (≈7 операций PyTorch)
rope_lora_kernel	RoPE + сложение с LoRA
rope_inverse_kernel	backward RoPE
masked_rows_kernel	dropout LoRA
masked_rows_bwd_kernel	backward dropout + сложение градиентов
gated_silu_q8_kernel	SiLU·up forward, int8 для backward
gated_silu_q8_bwd_kernel	SiLU·up backward из int8

Матмулы — cuBLAS, attention — kernel PyTorch: их своими не считаем. Без torch.compile и CUDA graphs.

Ограничения и что дальше

Ограничения

- Распаковка NF4 дважды за шаг — ~10 % GPU
- GPU простаивает ~15–20 % шага, пока Python раздаёт мелкие операции
- int8-активации — шум в градиентах (выключается)
- Быстрый путь — под эту модель; иначе исходный код

Следующие шаги

- GEMM с распаковкой NF4 внутри K-цикла
- Выгрузка активаций и весов в RAM с подкачкой параллельно вычислениям
- Слияние распаковки q|k|v и gate|up в один запуск
- Битовые маски dropout вместо повторной генерации

AGENTIC_REMAIDER

Спасибо!

1,80× быстрее · 5,17× меньше памяти · 10/10 проверок

Код: github.com/BAITC-Hacks/hack-5f644d36-agentic-remaider

Kaggle: kaggle.com/code/abekdr/notebookd836800e9b